



Understanding RESTful APIs

Welcome to the essential guide for mobile and web developers. This presentation will demystify the core components of RESTful APIs, empowering you to build robust and efficient applications.

The Building Blocks of an API

1

Base URL

The foundational address for all server interactions.

2

Endpoint

Specific paths defining actions or resources on the server.

3

HTTP Methods

The verbs that specify the type of operation (GET, POST, PUT, DELETE).

4

Parameters

Data sent with requests to customize actions or filter results.

5

Headers

Additional metadata for authentication and content type.



1. The Base URL: Your API's Home Address

Universal Starting Point

- It's the common prefix for every request you make to the server.
- Think of it as the main domain for your API services.

Why It Matters

Storing the Base URL in a constant makes your code **maintainable**. If the API server's location changes, you only need to update it in one place, preventing widespread code modifications.

Example:

```
https://api.example.com
```

In Flutter:

```
class ApiConstants {  
  static const String  
    baseUrl =  
    'https://api.example.com';  
}
```

2. Endpoints: Specific Actions & Resources

Defining Specific Paths

Endpoints are appended to the Base URL, directing your request to a specific resource or action on the server. Each endpoint serves a distinct purpose.

Examples of common endpoints:

- `/login` → For user authentication
- `/users` → To retrieve user data
- `/products/1` → To fetch a specific product

Full URL Construction:

Combining the Base URL with an Endpoint creates a complete request URL:

```
https://api.example.com/login
```

This structure provides a clear, organized way to interact with various functionalities of an API.



3. HTTP Methods: The Verbs of Your Request

HTTP methods tell the server *what kind of operation* you want to perform on the specified endpoint.

GET	Used to retrieve data from the server. It's a read-only operation.
POST	Used to send new data to the server, typically to create a new resource.
PUT / PATCH	Used to update existing data on the server. PUT typically replaces the entire resource, while PATCH applies partial modifications.
DELETE	Used to remove data from the server.

For example, a `POST /login` request would send user credentials to authenticate.



4. Parameters: Customizing Your Requests

Parameters are crucial for sending dynamic information with your API requests, allowing for flexible and targeted interactions.

Query Parameters

Appended to the URL with a question mark (?), typically for GET requests. Ideal for filtering, pagination, and searching.

```
/products?  
page=1&limit=10
```

Path Parameters

Integrated directly into the endpoint's path, these identify specific resources.

```
/products/5
```

(Fetches product with ID = 5)

Body Parameters

Sent in the request body, usually with POST or PUT methods, for creating or updating data.

```
{ "email":  
  "user@example.com",  
  "password": "secure" }
```


5. Headers: Providing Context & Security

Headers are key-value pairs that carry metadata about the request or response. They don't affect the resource itself but influence how the server processes the request.

Content-Type

Informs the server about the format of the data being sent in the request body (e.g., `application/json`).

Authorization

Carries authentication credentials (e.g., `Bearer TOKEN`) to verify the client's identity and permissions.

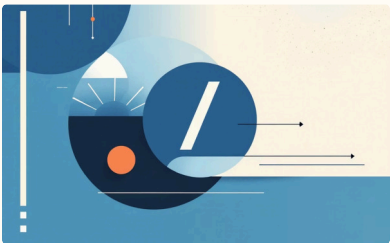
Headers are essential for ensuring secure communication and correct data interpretation between your client and the API.



CHAPTER 2

Networking Packages in Flutter

Flutter offers excellent tools for API communication. Let's explore two popular choices for handling your networking needs.



The http Package

A lightweight and official Dart package, ideal for beginners and smaller projects.



The Dio Package

A feature-rich and robust solution, preferred for professional-grade applications.

Flutter's http Package: Simple & Efficient

Key Features:

- **Simplicity:** Easy to understand and use, making it perfect for those new to Flutter networking.
- **Lightweight:** Minimal overhead, suitable for basic API interactions.
- **Manual Handling:** Requires more manual management for error handling and complex requests.

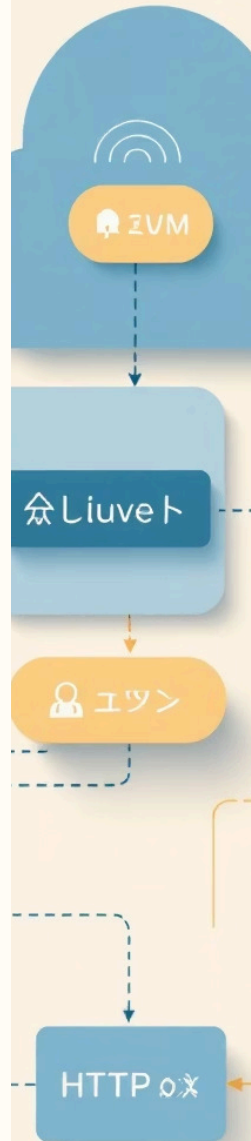
📄 **Best Use Case:** Quick prototypes, learning API concepts, or applications with straightforward networking requirements.

Example GET Request:

```
import 'package:http/http.dart' as http;

final response = await http.get(
  Uri.parse('$baseUrl/users'),
);

if (response.statusCode == 200) {
  // Process data
} else {
  // Handle error
}
```





Flutter's Dio Package: Power & Flexibility

Advanced Capabilities:

- **Interceptors:** Automatically modify requests/responses (e.g., adding headers, logging).
- **Robust Error Handling:** Centralized and more sophisticated error management.
- **Request Options:** Supports timeouts, retries, and cancellation.
- **File Operations:** Built-in support for FormData, file uploads, and downloads.

📋 **Ideal For:** Large-scale applications, complex API architectures, and seamless integration with state management like BLoC/Cubit.

Example POST Request:

```
import 'package:dio/dio.dart';

Dio dio = Dio();
final response = await dio.post(
  '$baseUrl/login',
  data: {
    "email": "user@example.com",
    "password": "securepassword"
  },
);

if (response.statusCode == 200) {
  // Handle successful login
} else {
  // Handle specific DioError
}
```